

SERC Mini-OS: Technical Report
Operating System Simulation

LECTURER: DR Ntalasha

Group Members

NAME	SIN
Emmanuel Nkaka	23130711
Paul Banda	22103558
Luckson Chama	22106051
Mobe Chamululu	24160351

Table Of Content

1) Introduction.....2
1.1) Objectives.....3
2) Development Steps 4
3) System Architecture and Data Structures 4
4) Algorithms Implemented.....6
4.1) Creating the GUI with GTK 8
4.2) Creating the Installer with Inno Setup.....9
5) **Screenshots of Execution**.....12
6) Expected Output.....17
7) Challenges 17
8) Current Limitations.....18
9) Conclusion 19
10) Quantitative Results.....19

1) Introduction

Operating systems are fundamental to modern computing, yet their internal workings remain abstract and difficult for students to grasp through theoretical study alone. Traditional textbook explanations of process scheduling, memory management, deadlock avoidance, and inter-process communication often fail to convey the dynamic interactions between these subsystems.

Mini-OS Monitor is a tool that simulates core OS components including process management, CPU scheduling, memory allocation, deadlock prevention via Banker's algorithm, and inter-process communication mechanisms. Built with the C programming language and GTK+3 graphical toolkit, the application provides both command-line feedback and rich visualizations such as Gantt charts for scheduling and block diagrams for memory mapping.

The system is designed for use in operating systems courses, allowing instructors to demonstrate algorithms in real-time and experiment with different scheduling policies, resource allocation strategies, and IPC methods. By providing immediate visual feedback and performance metrics.

1.1) Objectives

The Mini-OS for SERC following operating system components:

Table: Checklist

Component	Implementation Status
Process Management	
CPU Scheduling (2+ algorithms)	FCFS, Round Robin
Memory Management	First-Fit
Inter-Process Communication	Messages, Pipes, Shared Memory
Deadlock Handling	Banker's Algorithm
File Management	Event Logging

.

2) Development Steps

Prerequisites

- **C Compiler** (GCC/MinGW or MSYS2)
- **GTK Library** (Download and set up)
- **Inno Setup** (For creating the installer)
- **Code Editor** (Code blocks)
- **GCC Compiler 9.0+**
- **GTK+ 3.0**
- **Cairo Graphics** (Gantt chart rendering)

3) System Architecture and Data Structures

PRESENTATION LAYER (GTK+3)

- Create Task
- List Task
- FCFS
- Round Robin
- Banker Algorithms

CORE OS SIMULATION LAYER

- Process Manager (pcb.c)
- Memory Manager (memory.c)
- Scheduler (scheduler.c)
- Banker (banker.c)
- IPC system (ipc.c)
- Logger system (logger.c)
- Memory map (memory_map.c)

Process Control Block (PCB)

```
typedef struct {
    int pid;                // Unique Process ID
    char name[MAX_NAME_LEN];    // Task name: Ambulance, Fire, Police
    int priority;           // 1=Highest(Ambulance), 2=High(Fire), 3=Normal(Police)
    int burst_time;         // Total CPU time needed (ms)
    int remaining_time;     // Remaining CPU time
    int arrival_time;       // Emergency arrival time
    proc_state_t state;     // NEW, READY, RUNNING, WAITING, TERMINATED
    int mem_addr;           // Allocated memory address
```

Memory Block (Linked List for First-Fit)

```
typedef struct mem_block {
    int start;              // Starting address in KB
    int size;               // Block size in KB
    int free;               // 1 = free, 0 = allocated
    struct mem_block *next;     // Pointer to next block
} mem_block_t;    int mem_size;           // Memory size allocated (KB)
} pcb_t;
```

Banker's Algorithm Tables

```
static int total[3];       // Total resources: Channels, Vehicles, Personnel
static int available[3];    // Currently available resources
static int max_claim[64][3]; // Maximum claim per process
static int allocation[64][3];// Currently allocated to each process
static int need[64][3];     // Remaining need = max - allocation
```

4) Algorithms Implemented

First Come First Serve (FCFS) Scheduling

```
Algorithm FCFS_Scheduler(processes):
    time = 0
    for each process in order of arrival:
        process.state = RUNNING
        record Gantt segment (process.pid, time, process.burst_time)
        time = time + process.burst_time
        process.completion_time = time
        process.turnaround_time = completion_time - arrival_time
        process.waiting_time = turnaround_time - burst_time
        process.state = TERMINATED
    return time
```

Round Robin (RR) Scheduling

```
Algorithm RR_Scheduler(processes, quantum):
    time = 0
    while not all processes complete:
        for each process in ready queue:
            if process.remaining_time > 0:
                execute_time = min(process.remaining_time, quantum)
                record Gantt segment (process.pid, time, execute_time)
                process.remaining_time -= execute_time
                time += execute_time
                if process.remaining_time == 0:
                    process.completion_time = time
                    process.turnaround_time = completion_time - arrival_time
                    process.waiting_time = turnaround_time - burst_time
                else:
                    add process to end of ready queue
    return time
```

Banker's Algorithm (Deadlock Avoidance)

```
Algorithm Safety_Check():
    work[] = available[]
    finish[] = false for all processes

    repeat:
        changed = false
        for each process i with finish[i] = false:
            if need[i] <= work:
                work = work + allocation[i]
                finish[i] = true
                add i to safe_sequence
                changed = true
    until not changed

    return all finish[i] == true
```

4.1) Creating the GUI with GTK

We will use **GTK** to build a professional interface where users can:

- **Input emergency tasks** (Task Type, Arrival Time, Burst Time, Memory Required)
- **Set the time quantum** for Round Robin scheduling
- **View scheduling results** with Gantt chart visualization
- **View process tables, memory maps, and resource allocation status**
- **Save results to log file** for post-incident analysis

ScreenShot

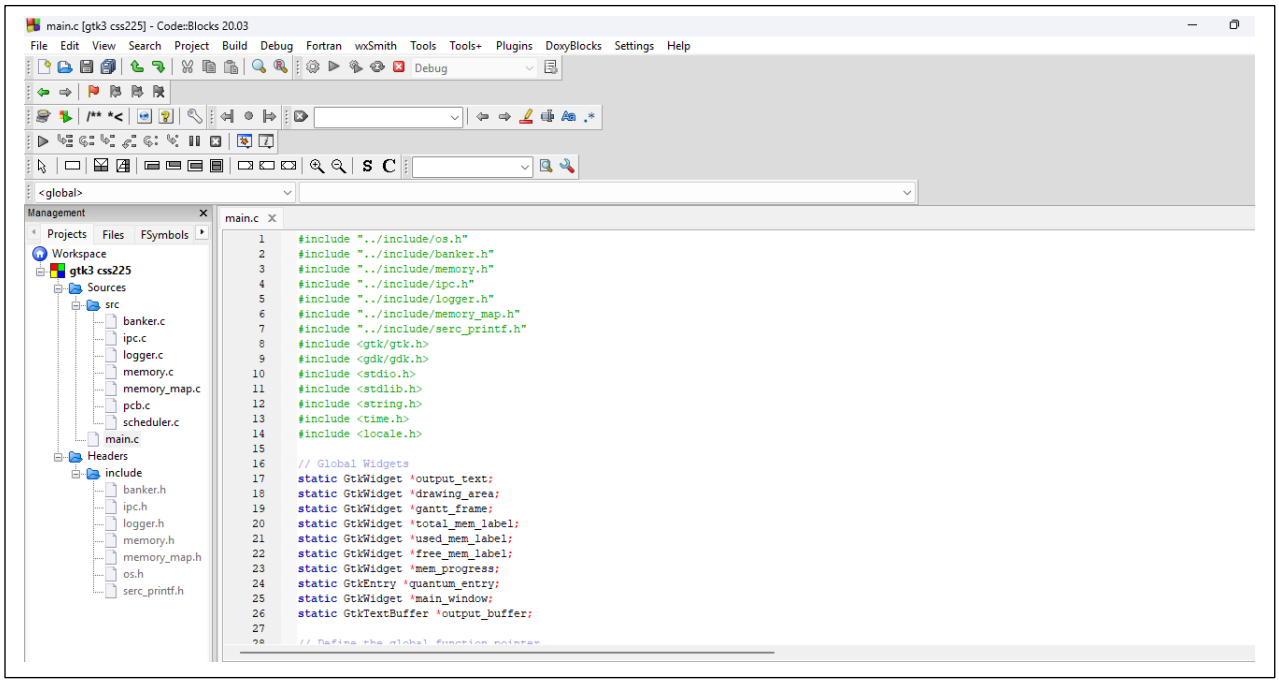


Figure 1.1: Codeblocks editor using gtk-3.0.

4.2) Creating the Installer with Inno Setup

1. Install Inno Setup

1. **Download** from https://jrsoftware.org/isinfo.php.
2. **Run the installer** and follow the setup wizard.

---

2. Prepare the Application Files

Before creating the installer, ensure:

- The compiled RoundRobinScheduler.exe is ready.
- Any required **DLLs** (e.g., GTK dependencies) are included. • Optional: Add an **application icon (.ico)**

3. Create an Inno Setup Script (.iss)

Inno Setup uses a **script file (**setup.iss**)** to define how the installer behaves.

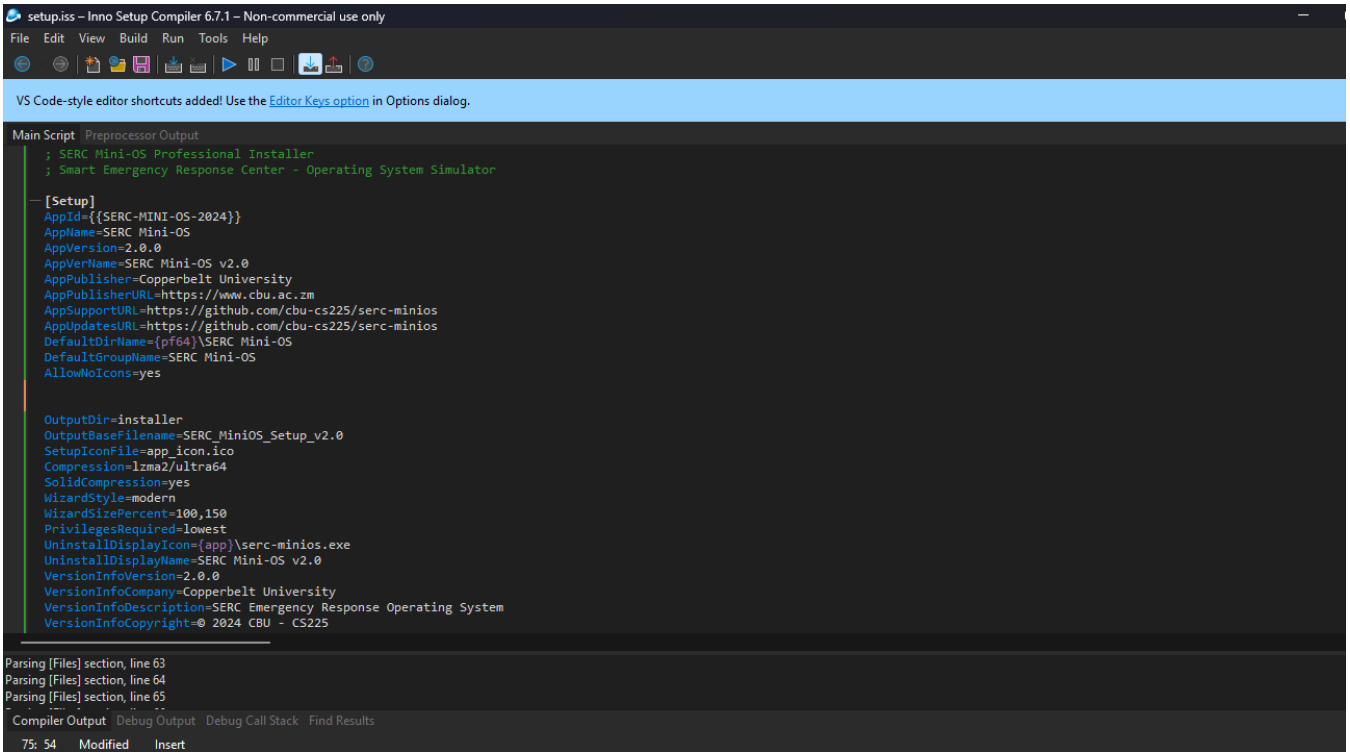
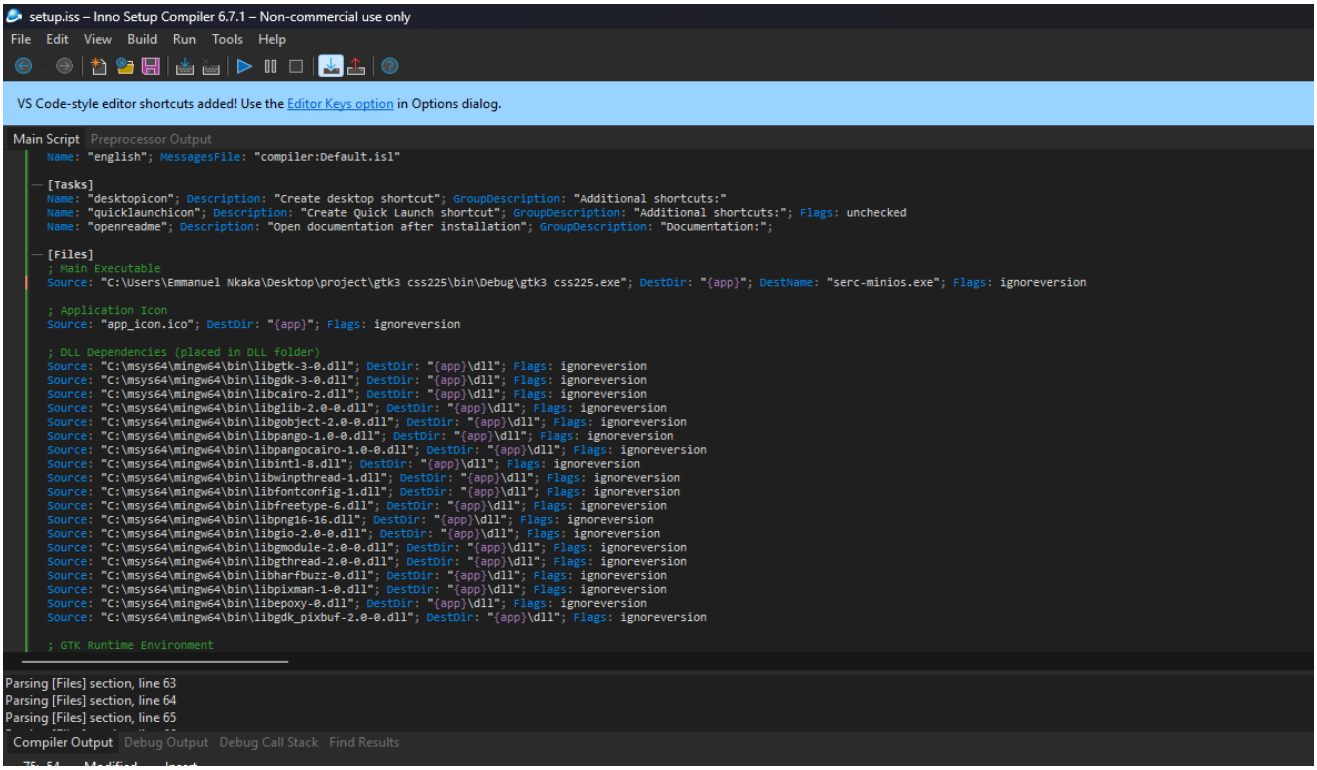


Figure 1.0.1: inno setup configurations.



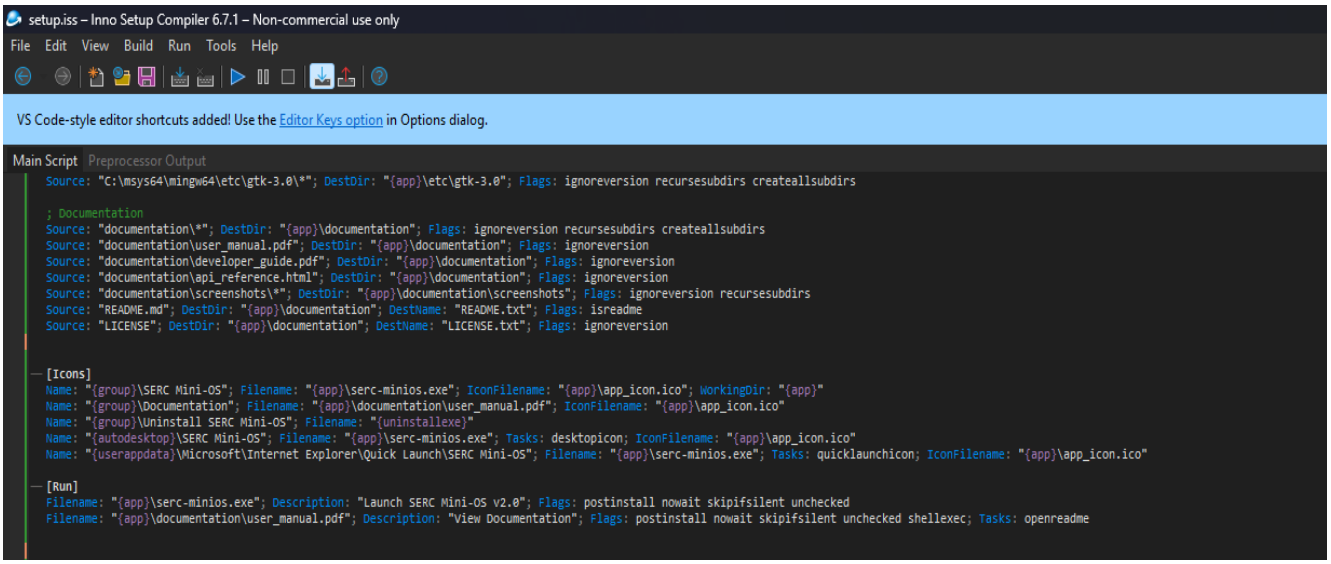


Figure 1.0.2 and 1.0.3: inno setup configurations.

Explanation of Key Sections

Section	Purpose	Key Parameters
[Setup]	General installer settings	AppName, AppVersion, DefaultDirName, OutputBaseFilename
[Files]	Specifies which files to install	Source (file path), DestDir (install location)
[Icons]	Creates Start Menu & Desktop shortcuts	{group} (Start Menu), {commondesktop} (Desktop)
[Run]	Optional: Run app after installation	postinstall nowait (doesn't block installer)

Compile the Installer

1. Open the .iss script in **Inno Setup Compiler**.
2. Click **Build → Compile**
3. The compiled installer (RoundRobinSchedulerSetup.exe)

5) Screenshots of Execution

Main Application Window

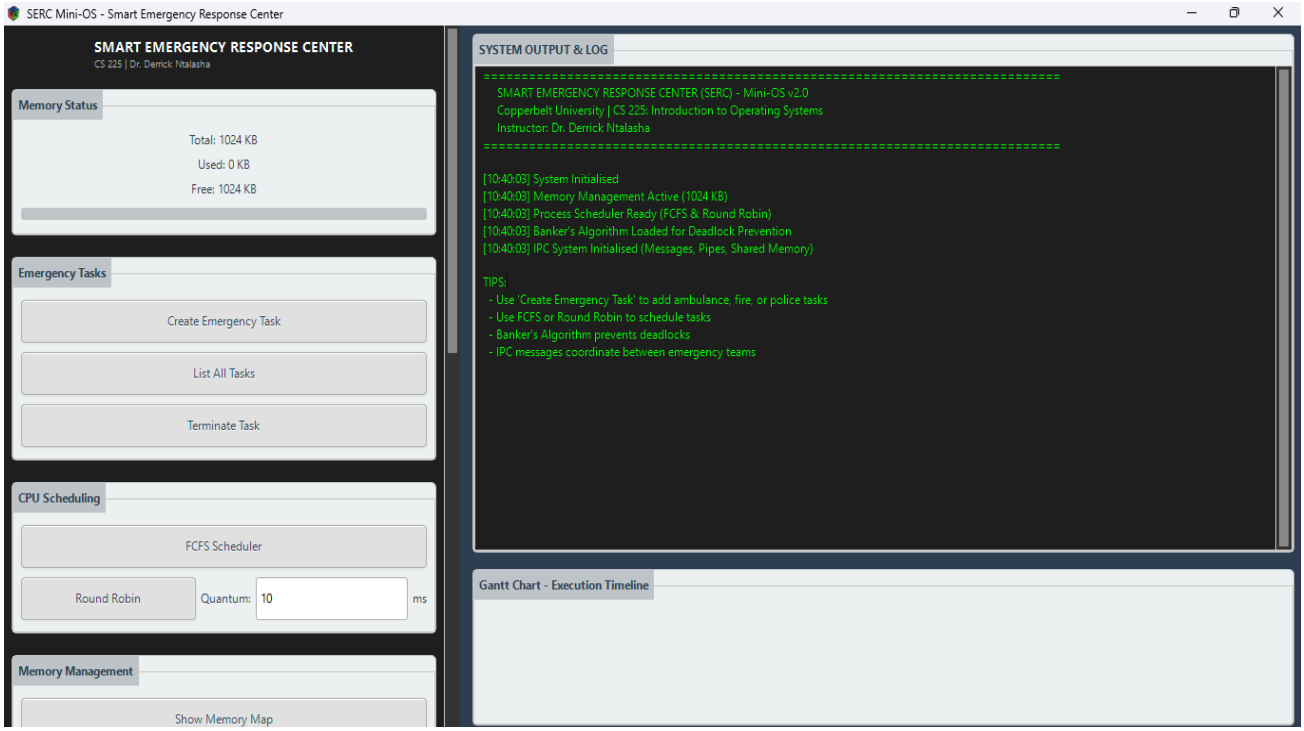


Figure 1.1.1: Smart Emergency Response Center start window.

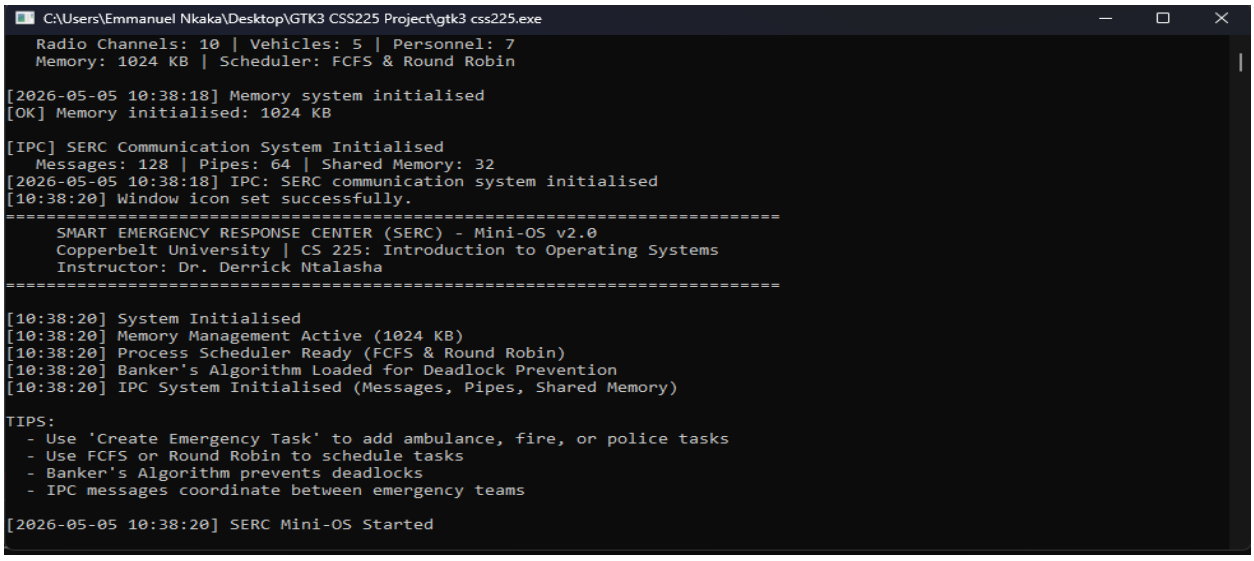


Figure 1.1.2: CMD terminal alongside working exe application.

FCFS Scheduler Output

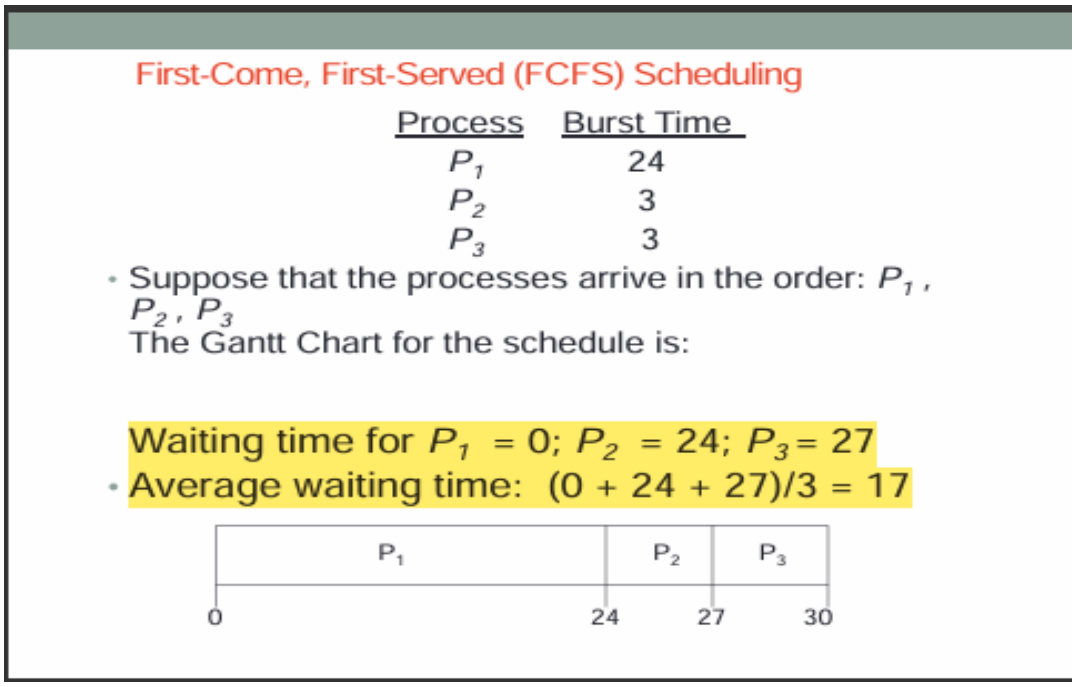


Figure 1.2 : Example of First-Come, First-Served (FCFS) Scheduling with reference from notes(CS225 UNIT 5).

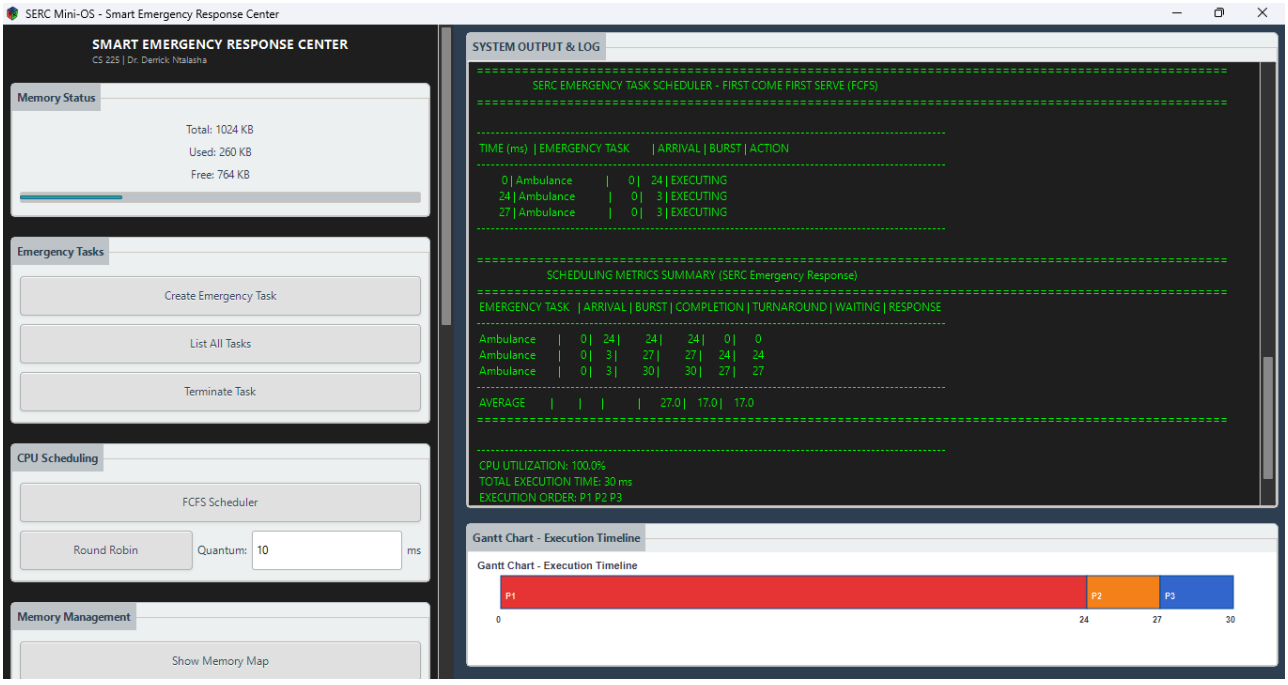


Figure 1.2.1: FCFS with output tab, Processes in this case are emergency tasks.

Round Robin

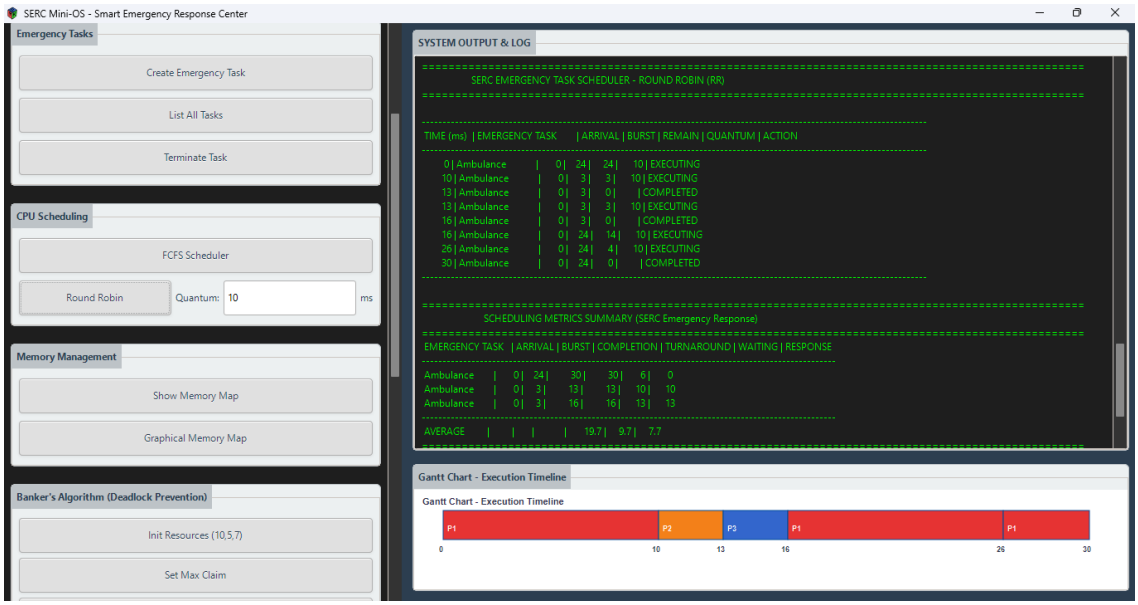


Figure 1.3.1: Round robin scheduling, with quantum of 10ms.

Banker's Algorithm Resource Status

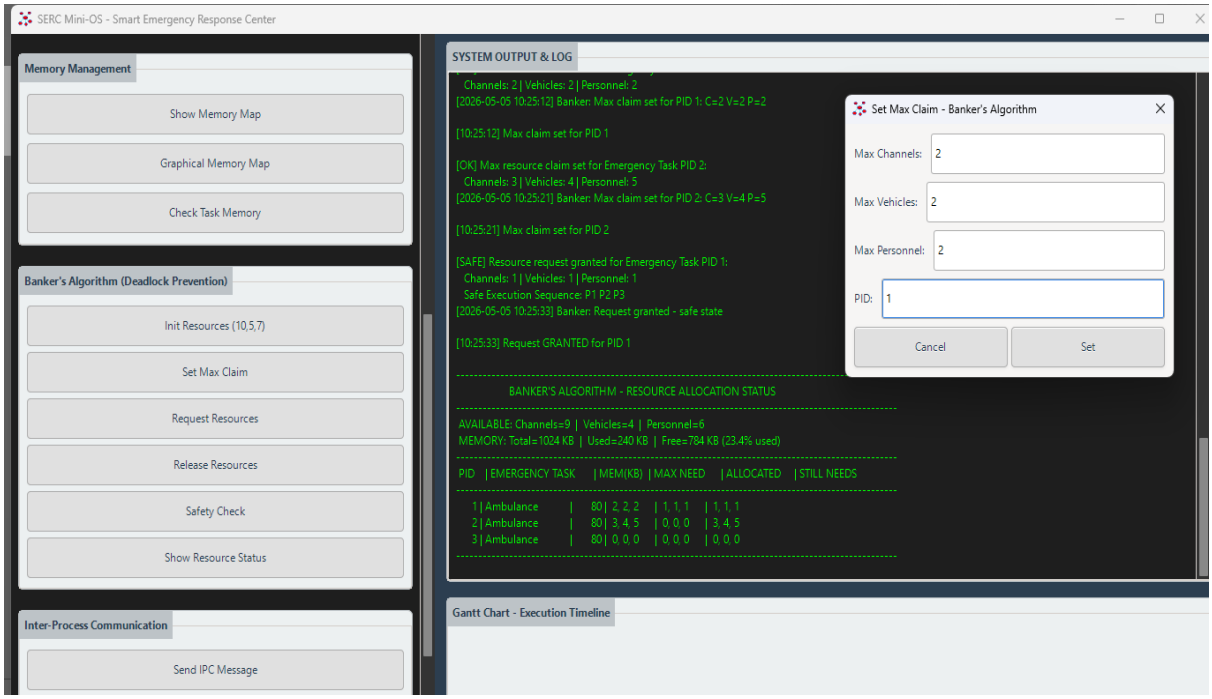


Figure 1.3.2: max needed memory set by clicking 'SET MAX CLAIM'

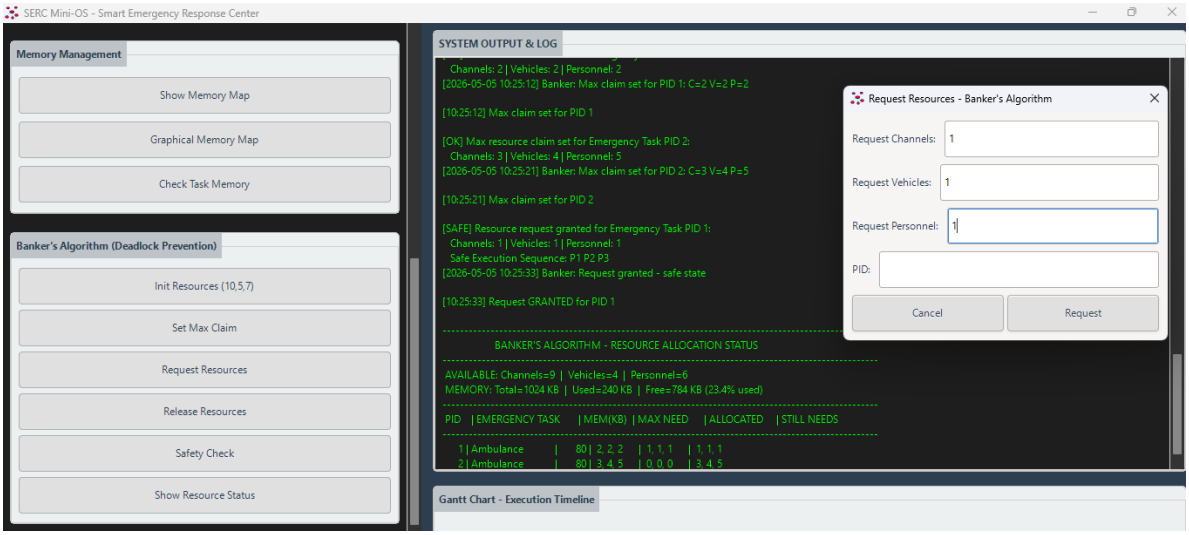


Figure 1.3.3: Request resources, setting process allocated memory set by clicking 'REQUEST RESOURCES'

Memory Map Display

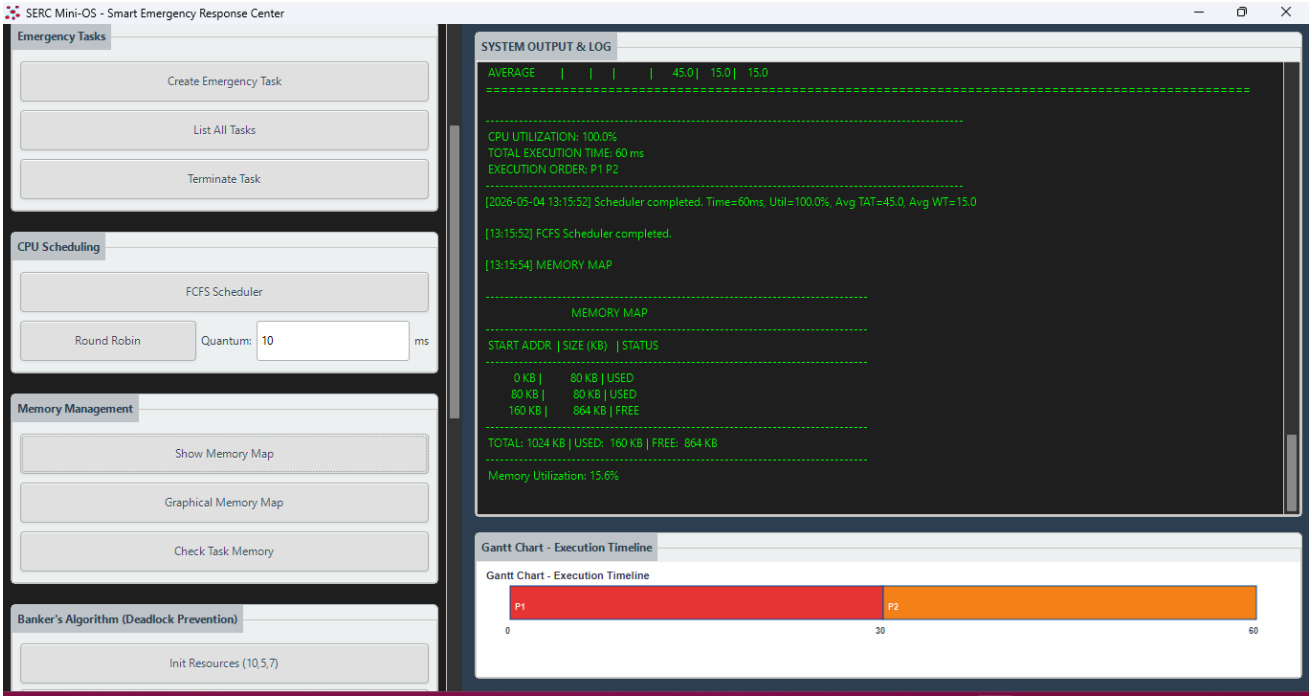


Figure 1.4: Memory display in system output and log.

Event Log Output

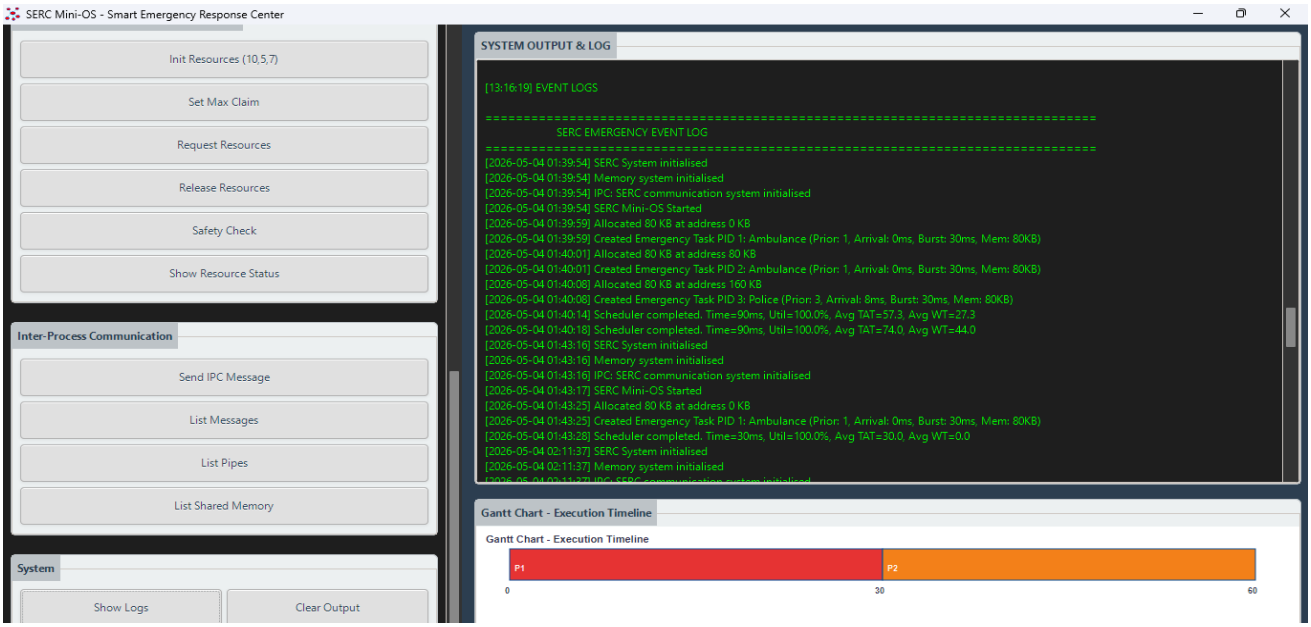


Figure 1.5: Event log in system output and log.

6) Expected Output

- A Windows executable (.exe) with a GUI
- An installer (.exe) that deploys the application.
- Creating Emergency Tasks
- List All Tasks Output
- FCFS Scheduler Output
- Round Robin Scheduler Output
- Memory Map Output
- Banker's Algorithm Output
- Requesting Resources (Safe/unsafe)
- Check Task Memory Output
- IPC Output

7) Challenges

- **Scrollbar Not Appearing,** Configured GTK_POLICY_ALWAYS for vertical scrollbar
- **Banker algorithm (allocation button),** Could not implement allocation

8) Current Limitations

Single-threaded Execution: All processes are simulated; no actual parallel execution

Fixed Resource Types: Banker's algorithm supports exactly 3 resource types

No Preemptive Priority Scheduling: Only FCFS and Round Robin implemented

Static Process Limit: Maximum 64 processes

No Virtual Memory: Simple contiguous allocation only

No Disk I/O Simulation: No secondary storage management

Fixed Memory Size: 1024 KB hardcoded, not configurable at runtime

9) Conclusion

The SERC Mini-OS successfully demonstrates the core concepts of operating systems through an interactive, visual simulation environment tailored for emergency response scenarios. All stated objectives were achieved

10) Quantitative Results

Codebase Size: 800 lines of C code (main.c file)

Supported Scheduling Algorithms: 2 (FCFS, Round Robin)

IPC Methods: 3 (Messages, Pipes, Shared Memory)

Resource Types: 3 (Channels, Vehicles, Personnel)

Max Processes: 64

Memory Size: 1024 KB